

▼ K-Means Clustering

K-means is an unsupervised learning algorithm used to group similar data points into clusters.

Goal 📌

Divide data into K groups(clusters) where:

- points in the same cluster are similar
- Points in different clusters are different

How K-Means Works(steps)

1. Choose the number of clusters K
2. Randomly initialize K centroids
3. Assign each data point to the nearest centroid
4. Recalculate the centroid (mean of cluster points)
5. Repeat steps 3 and 4 until centroids stop changing

Cluster evaluation

1. Inertia (Within- Cluster Sum of Squares)

- Measure how tightly data points are grouped
- Sum of squared distances between points and their centroid
- Lower inertia = better clustering

2. Elbow Method

Used to find the best value of K.

- Plot : K vs Inertia
- Look for the elbow point where inertia stops decreasing sharply

That value of K is considered the best choice.

3. Sihouette Score

Measures how well clusters are separated.

range: -1 to +1

- +1 -> Perfect clustering
- 0 -> Overlapping
- -1 -> Wrong clustering

```
[1]: import warnings
warnings.filterwarnings("ignore")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

# 1. Load dataset
iris = load_iris()
X = iris.data
```

```
# 1. Load dataset
iris = load_iris()
X = iris.data

# 2. Feature scaling (important for K-Means)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Elbow Method (Inertia)

inertia = []
K_range = range(1, 11)
for k in K_range:

    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)

    kmeans.fit(X_scaled)

    inertia.append(kmeans.inertia_)

plt.plot(K_range, inertia, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of clusters (K)")
plt.ylabel("Inertia")
plt.show()

# 4. Fit final KMeans (choose K = 3 for iris)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)

# 5. Silhouette Score

score = silhouette_score(X_scaled, labels)
print("Silhouette Score:" score)
```

```
# 5. Silhouette Score

score = silhouette_score(X_scaled, labels)
print("Silhouette Score:", score)

# 6. Visualize clusters (first 2 features)

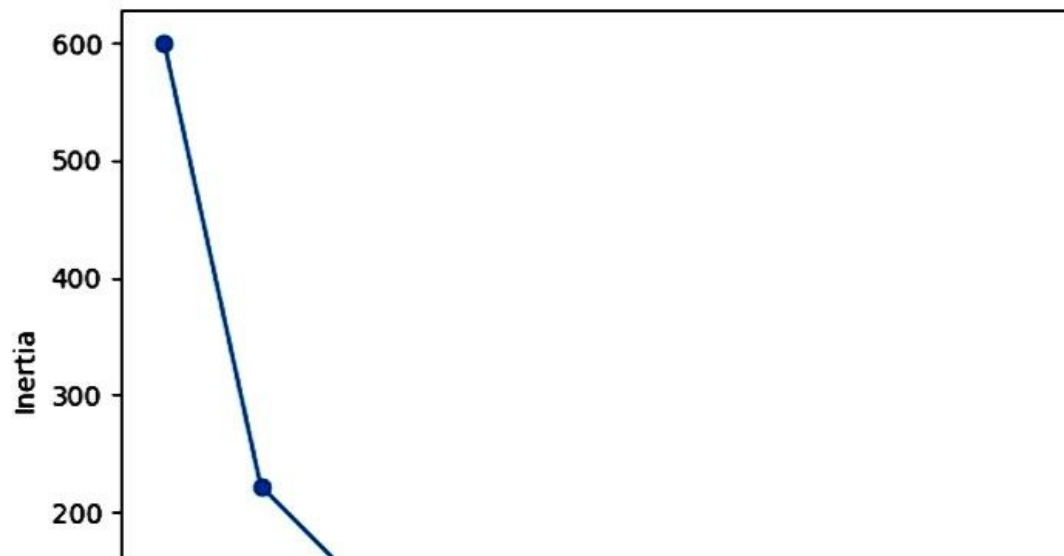
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap='viridis')
plt.scatter(kmeans.cluster_centers_[:, 0],

            kmeans.cluster_centers_[:, 1],

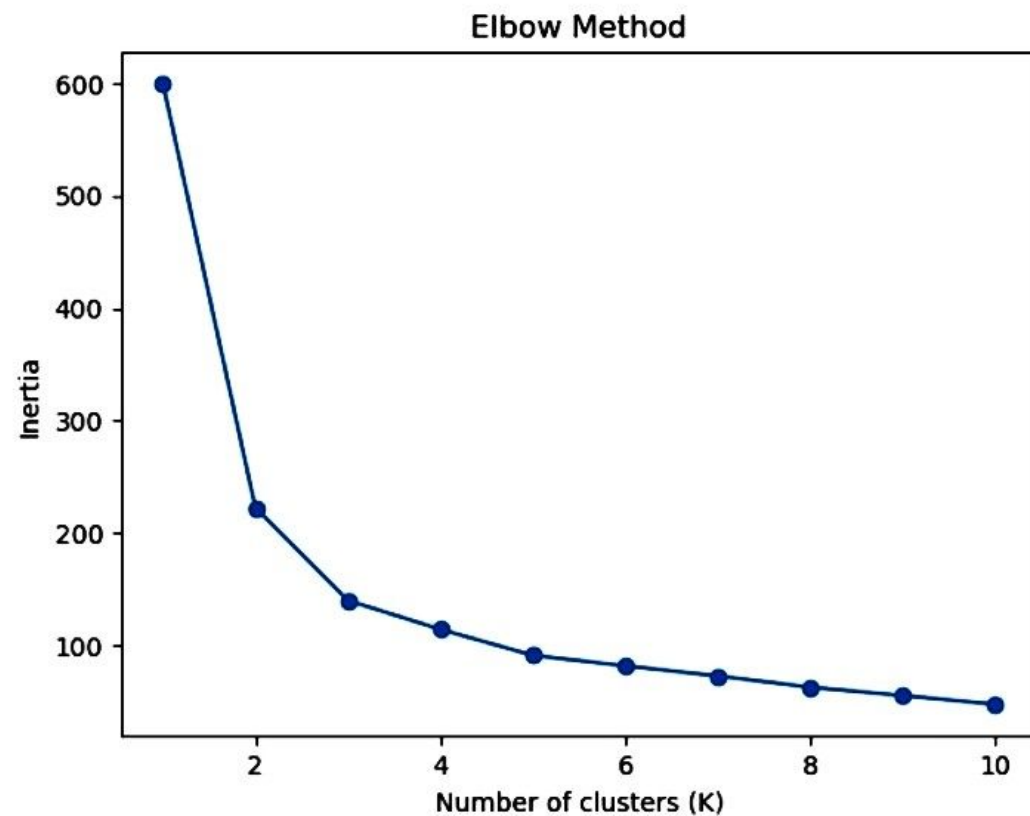
            s=300, c='red', marker='x')

plt.title("K-Means Clustering Result")
plt.show()
```

Elbow Method



```
plt.show()
```



Silhouette Score: 0.45994823920518635

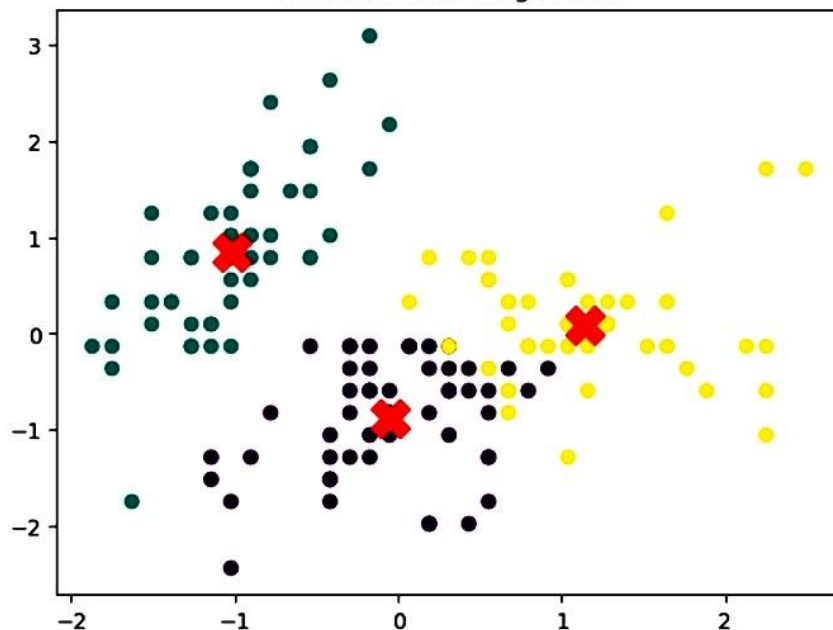
K-Means Clustering Result



Number of clusters (K)

Silhouette Score: 0.45994823920518635

K-Means Clustering Result



Hierarchical Clustering

Hierarchical Clustering is an unsupervised learning algorithm that groups similar data points into clusters by building a hierarchy of clusters.

Unlike K- Means you do not need to specify the no of clusters (K) initially.

▼ Hierarchical Clustering

Hierarchical Clustering is an unsupervised learning algorithm that groups similar data points into clusters by building a hierarchy of clusters.

Unlike K- Means you do not need to specify the no of clusters (K) initially.

How Hierarchical Clustering works?

There are two approaches:

1. Agglomerative(Bottom-UP)

- Start with each data point as its own cluster
- Merge the two closest clusters
- Repeat until all points belong to one cluster

2. Divisive(Top-Down)

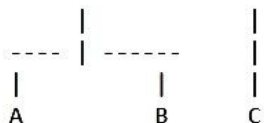
- Start with all data points in one cluster
- Split clusters recursively
- Continue until each point forms its own cluster

Dendrogram

A Dendrogram is a tree-like diagram that shows how clusters are merged.

- Each leaf represents a data point
- Branches show cluster merging
- Height represent distance between clusters

Example



- A and B merge first
- Then C joins
- Height indicates similarity

Linkage Methods

Linkage determines how the distance between clusters is calculated.

Single Linkage

Uses the minimum distance between clusters Distance = Minimum distance between any two points

Pros

- Finds irregular- shaped clusters

Cons

- Sensitive to noise

2. Complete Linkage

Uses the maximum distance between clusters.

Uses the maximum distance between clusters.

Distance = Maximum distance between any two points.

Pros

- Creates compact clusters

Cons

- Sensitive to outliers

3. Average Linkage

Uses the average distance between all points.

Distance = Average pairwise distance

Pros

- Balanced clustering

4. Ward Linkage (Most popular)

Merges clusters that minimize variance

Pros

- Produces well balanced clusters
- Often gives the best results

Use the dendrogram

Rule

1. Draw a horizontal line
2. Count how many vertical branches it crosses

```

      |
-----|----- ||
---- |----- ||| A B C

```

- If a horizontal line crosses 3 branches:
- No of clusters = 3
- Best practice
- Choose the cut where the vertical distance is largest.

```

[2]: import matplotlib.pyplot as plt
      from sklearn.datasets import load_iris
      from sklearn.preprocessing import StandardScaler
      from scipy.cluster.hierarchy import dendrogram, linkage
      from sklearn.cluster import AgglomerativeClustering

      # Load dataset
      iris = load_iris()
      X = iris.data

      # Scale features
      scaler = StandardScaler()

```

```
from sklearn.cluster import AgglomerativeClustering
```

```
# Load dataset
```

```
iris = load_iris()
```

```
X = iris.data
```

```
# Scale features
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
# Create linkage matrix
```

```
linked = linkage(X_scaled, method='ward')
```

```
# Plot dendrogram
```

```
plt.figure(figsize=(10,5))
```

```
dendrogram(linked)
```

```
plt.title("Dendrogram")
```

```
plt.xlabel("Samples")
```

```
plt.ylabel("Distance")
```

```
plt.show()
```

```
# Hierarchical Clustering
```

```
hc = AgglomerativeClustering(
```

```
    n_clusters=3,
```

```
    linkage='ward'
```

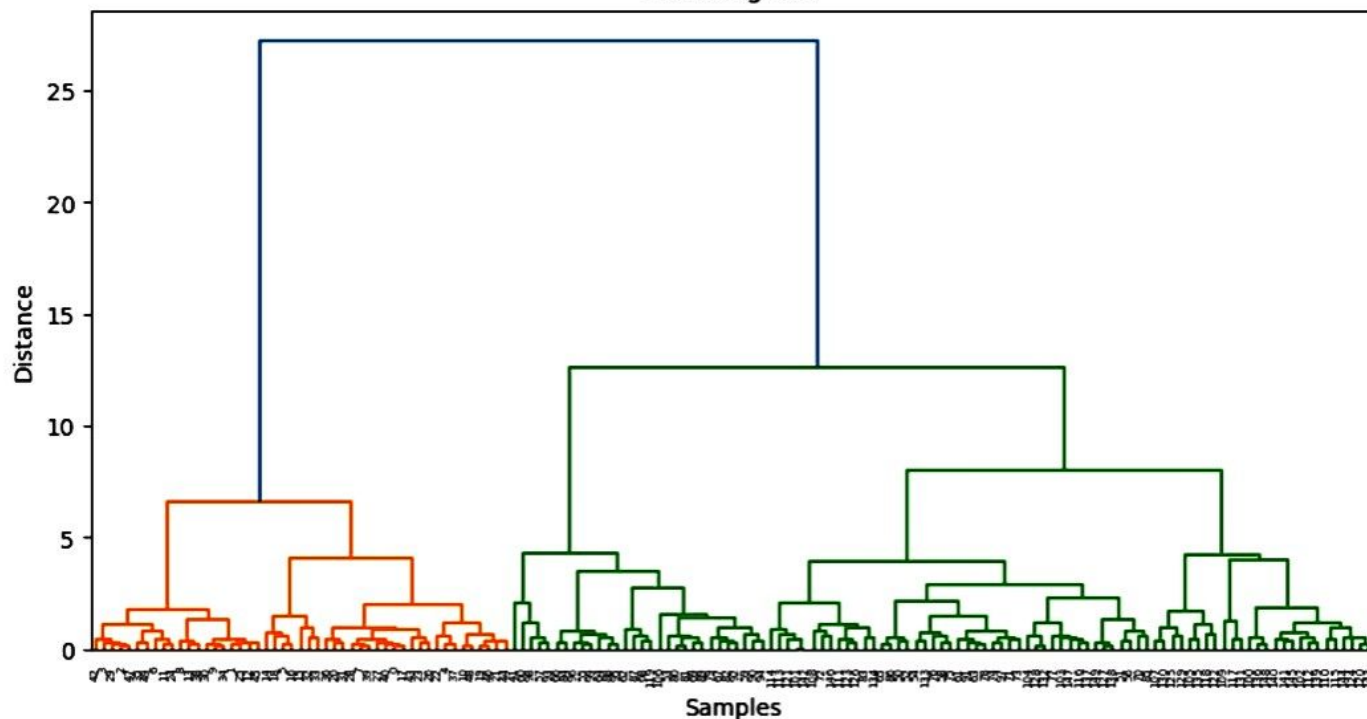
```
)
```

```
labels = hc.fit_predict(X_scaled)
```

```
print("Cluster Labels:")
```

```
print(labels)
```

Dendrogram



Cluster Labels:

```
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 2 1 1 1 1 1 1 1 1 1 0 0 0 2 0 2 0 2 0 2 0 2 0 2 2 2 2 0 0 0 0
 0 0 0 0 0 2 2 2 2 0 2 0 0 2 2 2 2 0 2 2 2 0 2 2 0 0 0 0 0 0 0 2 0 0 0 0
 0 0 0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0]
```

Multiple Choice Type Questions

Choose the correct option

20. What is the average time complexity for creating a set from an existing iterable, such as `set(l)`?

Negative marks: 0

Marks: 1

Answer

- ☐ A. $O(1)$
- ☐ B. $O(N \log N)$
- ☐ C. $O(\text{len}(l))$
- ☐ D. $O(N^2)$